

Environment Setup — MicroPython (ESP32-S3)

Environment Setup — MicroPython (ESP32-S3)

1. Overview
2. Preparation
 - 2.1 Hardware Requirements
 - 2.2 Items to Download
3. Install the CH340 Driver
 - 3.1 Check Whether the Driver Is Installed
 - 3.2 Download the Driver
 - 3.3 Installation Steps
 - 3.4 Verification
4. Flash the MicroPython Firmware
 - 4.1 Download the Firmware
 - 4.2 Install esptool
 - 4.3 Erase Flash
 - 4.4 Flash the Firmware
5. Install Thonny IDE
 - 5.1 Installation Steps
 - 5.2 Connect the ESP32-S3
6. Verification and the First Program
 - 6.1 REPL Interaction Verification
 - 6.2 Run the First Script
 - 6.3 File Management
7. Install Commonly Used Libraries
 - 7.1 Online Installation via mip
 - 7.2 Manually Upload .py Files
8. Common Problems

1. Overview

MicroPython is a streamlined implementation of Python 3 designed for microcontrollers. It can run `.py` scripts directly on the ESP32-S3 without compilation, making it well suited for rapid prototyping and teaching demonstrations.

This tutorial sets up the MicroPython development environment for the ESP32-S3 on **Windows**, in three steps:

1. **Flash the MicroPython firmware** — Write the `.bin` firmware into the ESP32-S3 Flash
2. **Install Thonny IDE** — A lightweight Python IDE with built-in MicroPython REPL support
3. **Verify REPL interaction** — Interact with the ESP32-S3 in real time through the REPL console

The MicroPython firmware is essentially a complete Python interpreter plus a set of hardware drivers. After flashing, the ESP32-S3 enters REPL mode on power-up, and you can type Python statements directly over the serial port to execute them.

2. Preparation

2.1 Hardware Requirements

Item	Requirement
Operating System	Windows 10 / 11 (64-bit)
Disk Space	≥ 500 MB (Thonny IDE + firmware files + project scripts)
Memory	≥ 4 GB
USB Port	At least 1 USB-A / USB-C port (to connect the development board)
Network	Access to micropython.org (to download the firmware)

2.2 Items to Download

File	Description	Download Address
MicroPython firmware	<code>.bin</code> firmware for the ESP32-S3	https://micropython.org/download/ESP32_GENERIC_S3/
Thonny IDE installer	Python IDE (with MicroPython support)	6.Software Tools → thonny

The **esptool** flashing tool is installed online via `pip`; no manual download is needed.

Firmware (Support for Octal-SPIRAM)

Releases

v1.28.0 (2026-04-06)	.uf2	[.app-bin]	[.bin]	[.elf]	[.map]	[Release notes]	(latest)
v1.27.0 (2025-12-09)	.uf2	[.app-bin]	[.bin]	[.elf and .map]	[.map]	[Release notes]	
v1.26.1 (2025-09-11)	.uf2	[.app-bin]	[.bin]	[.elf and .map]	[.map]	[Release notes]	
v1.26.0 (2025-08-09)	.uf2	[.app-bin]	[.bin]	[.elf and .map]	[.map]	[Release notes]	
v1.25.0 (2025-04-15)	.uf2	[.app-bin]	[.bin]	[.elf and .map]	[.map]	[Release notes]	
v1.24.1 (2024-11-29)	.uf2	[.app-bin]	[.bin]	[.elf and .map]	[.map]	[Release notes]	
v1.24.0 (2024-10-25)	.uf2	[.app-bin]	[.bin]	[.elf and .map]	[.map]	[Release notes]	
v1.23.0 (2024-06-02)	.uf2	[.app-bin]	[.bin]	[.elf and .map]	[.map]	[Release notes]	
v1.22.2 (2024-02-22)	.uf2	[.app-bin]	[.bin]	[.elf and .map]	[.map]	[Release notes]	
v1.22.1 (2024-01-05)	.uf2	[.app-bin]	[.bin]	[.elf and .map]	[.map]	[Release notes]	
v1.22.0 (2023-12-27)	.uf2	[.app-bin]	[.bin]	[.elf and .map]	[.map]	[Release notes]	
v1.21.0 (2023-10-05)	.uf2	[.app-bin]	[.bin]	[.elf and .map]	[.map]	[Release notes]	
v1.20.0 (2023-04-26)	.uf2	[.bin]	[.elf and .map]	[.elf and .map]	[.map]	[Release notes]	

3. Install the CH340 Driver

The ESP32-S3 development board communicates with the computer through a USB-to-serial chip. Most boards use the **CH340** series chip, whose driver is not included in Windows by default and must be installed manually.

3.1 Check Whether the Driver Is Installed

- 1. Connect the board's **COM port** to the computer with a USB data cable
- 2. Right-click **Start Menu** → **Device Manager** → expand **Ports (COM & LPT)**
- 3. Check the list:

Condition	Status	Action
COMx appears (no exclamation mark)	Driver ready	Skip this section
A device with a yellow exclamation mark appears	Driver missing	Continue installing
No new device appears	USB cable / port issue	Replace the data cable or USB port

3.2 Download the Driver

[Home - Nanjing Qinheng Microelectronics Co., Ltd.](#) → download the latest `CH340SER.EXE` (compatible with the CH340 series).

You can also search for "WCH CH340 driver" and download it from the official website. The board used in this tutorial uses the CH340K chip; the installer is about 300 KB.

3.3 Installation Steps

- 1. Double-click `CH341SER.EXE`
- 2. Click **INSTALL**
- 3. After installation completes, the prompt "**Driver install success!**" appears
- 4. Re-plug the board's USB cable

3.4 Verification

Go back to **Device Manager** → **Ports (COM & LPT)**, you should see:

```
USB-SERIAL CH340 (COM3)
```

Note down the port number (e.g., `COM3`); you will need to select it later when flashing the firmware and connecting Thonny.

4. Flash the MicroPython Firmware

4.1 Download the Firmware

1. Open the [MicroPython ESP32-S3 download page](#)
2. Download the latest **Stable** version of the `.bin` file (e.g., `ESP32_GENERIC_S3-SPIRAM_OCT-20260406-v1.28.0.bin`)

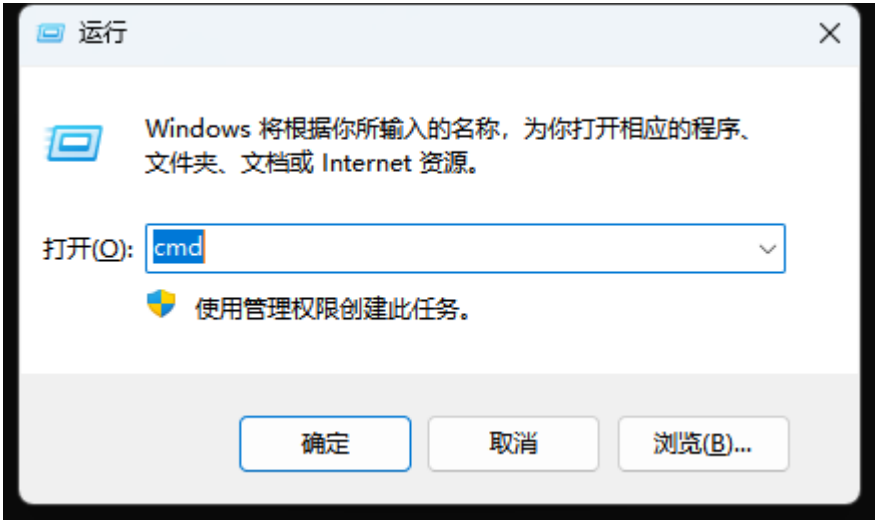
Firmware Type	Applicable Condition
ESP32_GENERIC_S3	Flash ≥ 8MB, no PSRAM or PSRAM ≤ 2MB
ESP32_GENERIC_S3_SPIRAM	With PSRAM (≥ 8MB), such as the ESP32-S3 N16R8 used in this tutorial

The board used in this tutorial is the **ESP32-S3 N16R8 (16MB Flash + 8MB PSRAM)**; select the `GENERIC_S3_SPIRAM` firmware.

4.2 Install esptool

`esptool` is Espressif's official Flash flashing tool, installed via Python's `pip`:

Open CMD (Win+R)



Download esptool

```
python -m pip install esptool
```

```
C:\Users\Administrator>python -m pip install esptool
Collecting esptool
  Using cached esptool-5.3.1-py3-none-any.whl
Requirement already satisfied: bitstring==4.2.0, >=3.1.6 in c:\users\administrator\appdata\local\programs\python\python312\lib\site-packages (from esptool) (4.4.0)
Requirement already satisfied: cryptography==43.0.0 in c:\users\administrator\appdata\local\programs\python\python312\lib\site-packages (from esptool) (49.0.0)
Requirement already satisfied: pyserial>=3.3 in c:\users\administrator\appdata\local\programs\python\python312\lib\site-packages (from esptool) (3.5)
Requirement already satisfied: reedsolomon==1.5.3 in c:\users\administrator\appdata\local\programs\python\python312\lib\site-packages (from esptool) (1.7.0)
Requirement already satisfied: PyVAVL>=5.1 in c:\users\administrator\appdata\local\programs\python\python312\lib\site-packages (from esptool) (6.0.3)
Requirement already satisfied: intelhex in c:\users\administrator\appdata\local\programs\python\python312\lib\site-packages (from esptool) (2.3.0)
Requirement already satisfied: rich_click<2 in c:\users\administrator\appdata\local\programs\python\python312\lib\site-packages (from esptool) (1.9.0)
Requirement already satisfied: click<9 in c:\users\administrator\appdata\local\programs\python\python312\lib\site-packages (from esptool) (8.4.2)
Requirement already satisfied: bitarray<4.0, >=3.0.0 in c:\users\administrator\appdata\local\programs\python\python312\lib\site-packages (from bitstring==4.2.0, >=3.1.6->esptool) (3.9.2)
Requirement already satisfied: tibs<0.6, >=0.5.6 in c:\users\administrator\appdata\local\programs\python\python312\lib\site-packages (from bitstring==4.2.0, >=3.1.6->esptool) (0.5.7)
Requirement already satisfied: colorama in c:\users\administrator\appdata\local\programs\python\python312\lib\site-packages (from click<9->esptool) (0.4.6)
Requirement already satisfied: cffi>=2.0.0 in c:\users\administrator\appdata\local\programs\python\python312\lib\site-packages (from cryptography==43.0.0->esptool) (2.1.0)
Requirement already satisfied: rich>=12 in c:\users\administrator\appdata\local\programs\python\python312\lib\site-packages (from rich_click<2->esptool) (15.0.0)
Requirement already satisfied: pycparser in c:\users\administrator\appdata\local\programs\python\python312\lib\site-packages (from cffi>=2.0.0->cryptography==43.0.0->esptool) (3.0)
Requirement already satisfied: markdown-it-py>=2.2.0 in c:\users\administrator\appdata\local\programs\python\python312\lib\site-packages (from rich>=12->rich_click<2->esptool) (4.2.0)
Requirement already satisfied: pygments<3.0.0, >=2.13.0 in c:\users\administrator\appdata\local\programs\python\python312\lib\site-packages (from rich>=12->rich_click<2->esptool) (2.20.0)
Requirement already satisfied: mdurl<=0.1 in c:\users\administrator\appdata\local\programs\python\python312\lib\site-packages (from markdown-it-py>=2.2.0->rich>=12->rich_click<2->esptool) (0.1.2)
Installing collected packages: esptool
Successfully installed esptool-5.3.1
```

Verify the installation:

```
esptool version
```

If you get a message that `esptool` is not recognized, install Python 3 (≥ 3.8) first and check "Add Python to PATH", then run `pip install esptool` again.

```
C:\Users\Administrator>esptool version
esptool v5.3.1
5.3.1
```

4.3 Erase Flash

Before flashing, erase the entire Flash first to avoid interference from old data:

```
esptool --chip esp32s3 --port COM3 erase-flash
```

Replace `COM3` with your actual port number. The erase process takes about 30 seconds to 2 minutes; wait for the prompt `Chip erase completed successfully`.

4.4 Flash the Firmware

You need the path to the downloaded firmware, for example: `D:\Micropython\ESP32_GENERIC_S3-SPIRAM_OCT-20260406-v1.28.0.bin`.

```
python -m esptool --chip esp32s3 --port COM8 write_flash -z 0x0
"D:\Micropython\ESP32_GENERIC_S3-SPIRAM_OCT-20260406-v1.28.0.bin"
```

Parameter	Description
<code>--chip esp32s3</code>	Target chip model
<code>--port COM8</code>	Serial port number (replace with your actual port)
<code>write-flash</code>	Write to Flash
<code>-z 0x0</code>	Firmware start address (ESP32-S3 starts at 0x0)

After a successful flash, the output looks similar to:

```
Connected to ESP32-S3 on COM8:
Chip type:          ESP32-S3 (QFN56) (revision v0.2)
Features:           Wi-Fi, BT 5 (LE), Dual Core + LP Core, 240MHz, Embedded PSRAM 8MB (AP_3v3)
Crystal frequency:  40MHz
MAC:               a4:cb:8f:f6:92:f8

Stub flasher running.

Configuring flash size...
Flash will be erased from 0x00000000 to 0x001adfff...
Wrote 1758064 bytes (1150760 compressed) at 0x00000000 in 101.8 seconds (138.1 kbit/s).
```

Hash of data verified.

Hard resetting via RTS pin...

If flashing fails with "Timed out waiting for packet header", you need to enter download mode manually:

Hold the BOOT key → press the RST key → release BOOT, then retry the flash command.

```
C:\Users\Administrator>python -m esptool --chip esp32s3 --port COM8 write_flash -z 0x0 "D:\Micropython\ESP32_GENERIC_S3-
SPIRAM_OCT-20260406-v1.28.0.bin"
Warning: Deprecated: Command 'write_flash' is deprecated. Use 'write-flash' instead.
esptool v5.3.1
Connected to ESP32-S3 on COM8:
Chip type:          ESP32-S3 (QFN56) (revision v0.2)
Features:           Wi-Fi, BT 5 (LE), Dual Core + LP Core, 240MHz, Embedded PSRAM 8MB (AP_3v3)
Crystal frequency: 40MHz
MAC:               a4:cb:8f:f6:92:f8

Stub flasher running.

Configuring flash size...
Flash will be erased from 0x00000000 to 0x001adfff...
Wrote 1758064 bytes (1150760 compressed) at 0x00000000 in 101.8 seconds (138.1 kbit/s).
Hash of data verified.

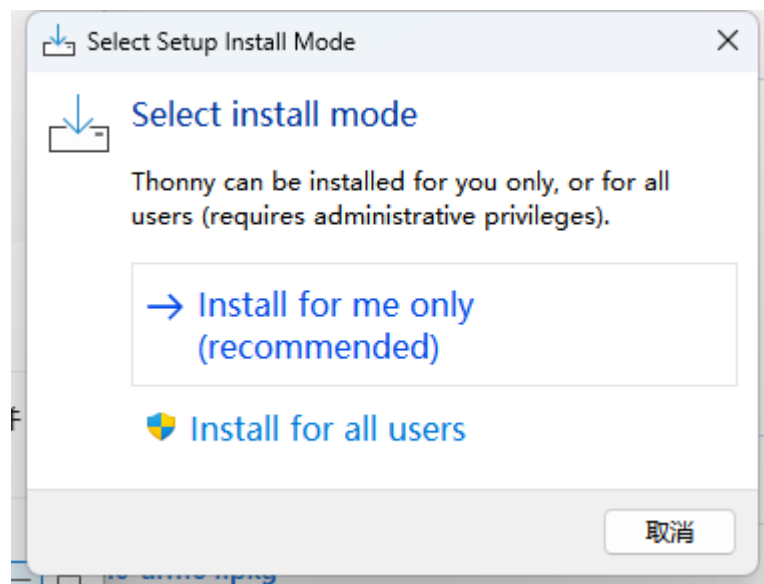
Hard resetting via RTS pin...
```

5. Install Thonny IDE

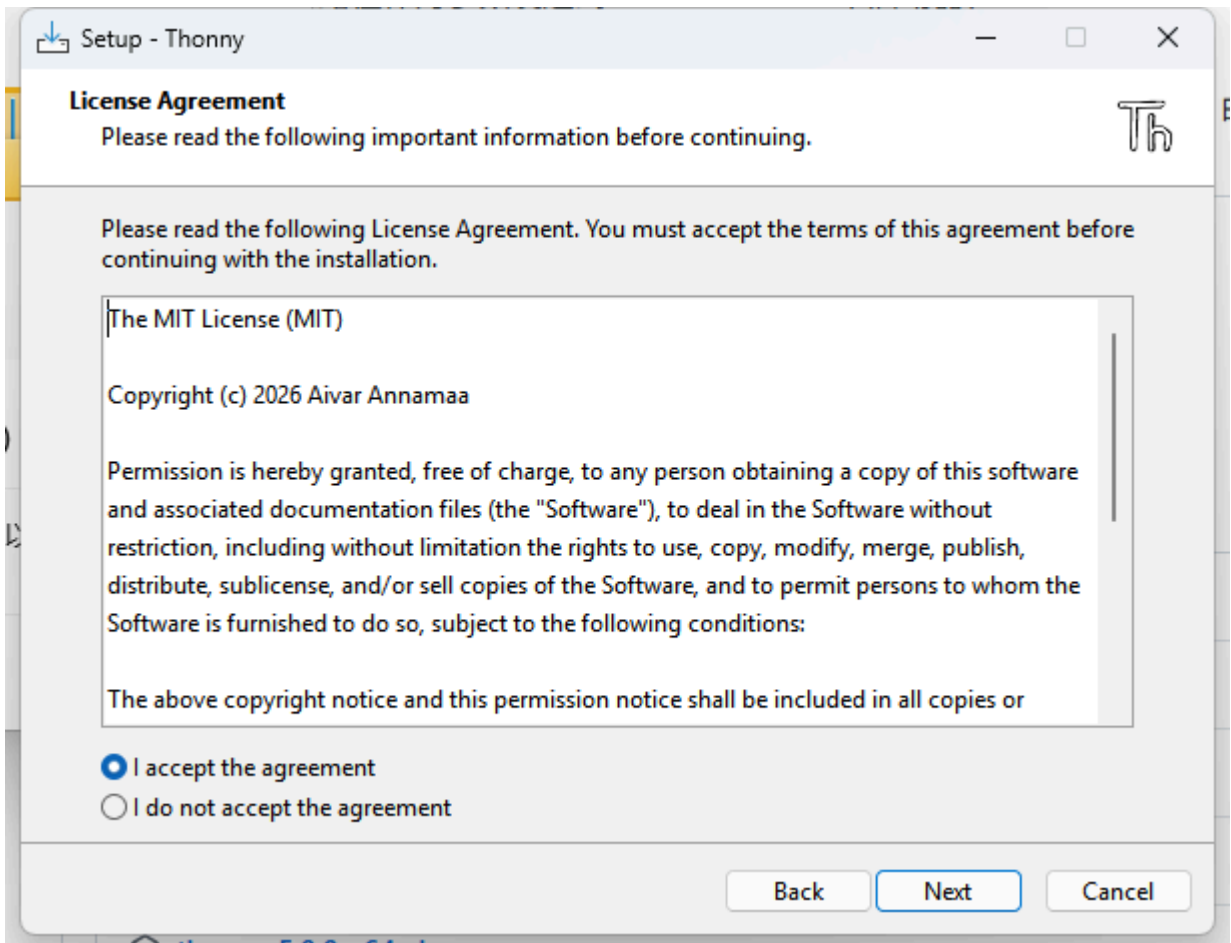
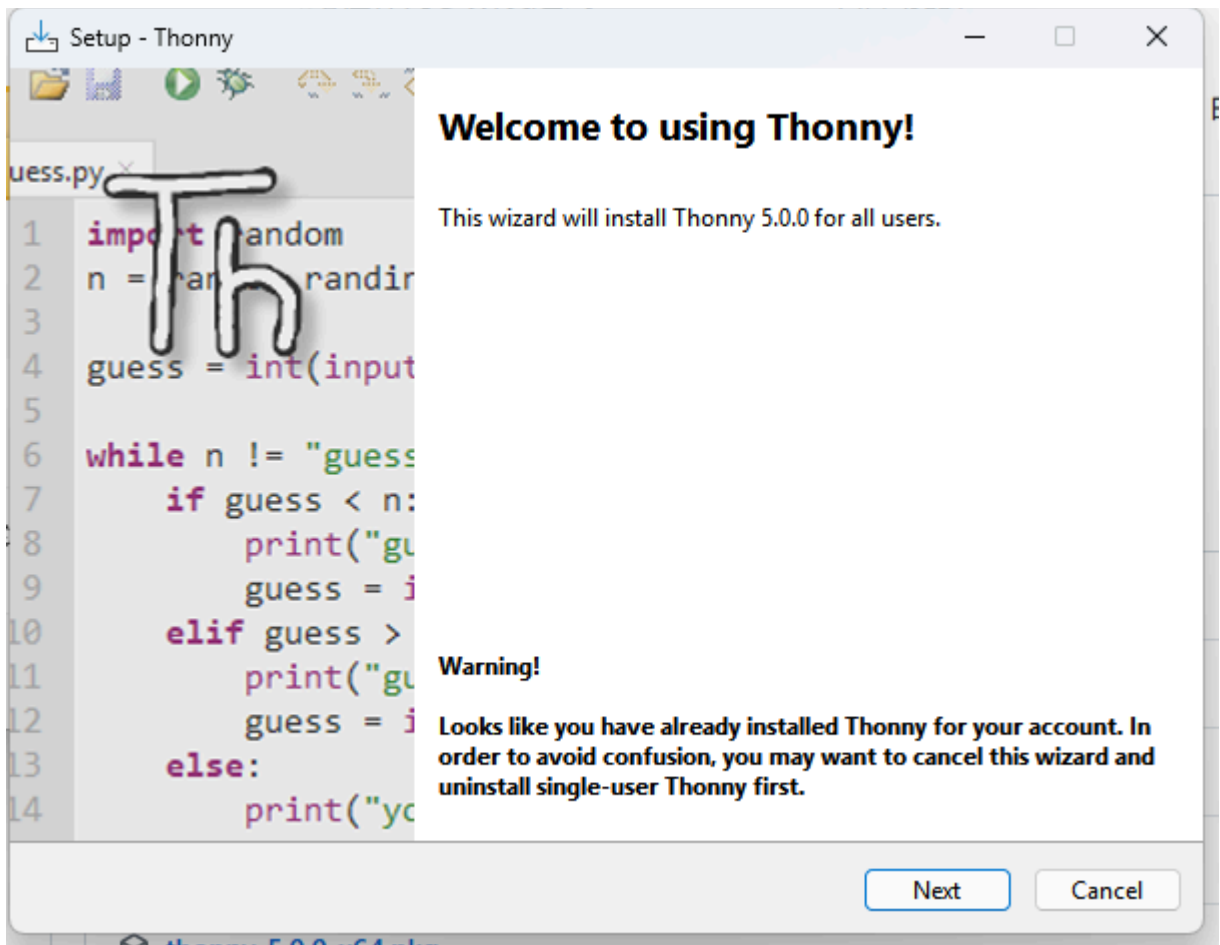
Thonny is a Python IDE designed for beginners, with built-in MicroPython REPL support and a file manager, requiring no additional plugin configuration.

5.1 Installation Steps

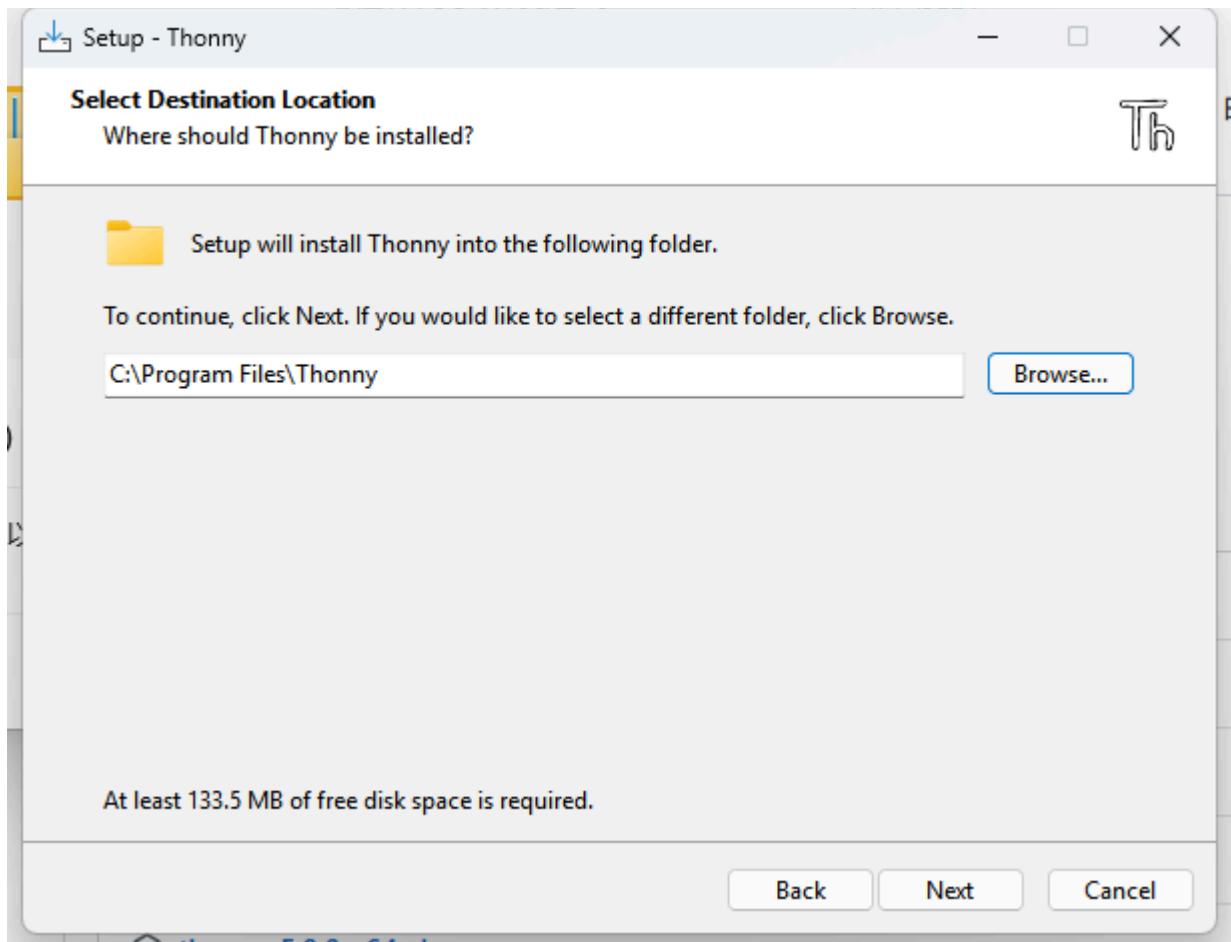
1. Double-click the Thonny installer (e.g., `thonny-5.0.0-x64.exe`)



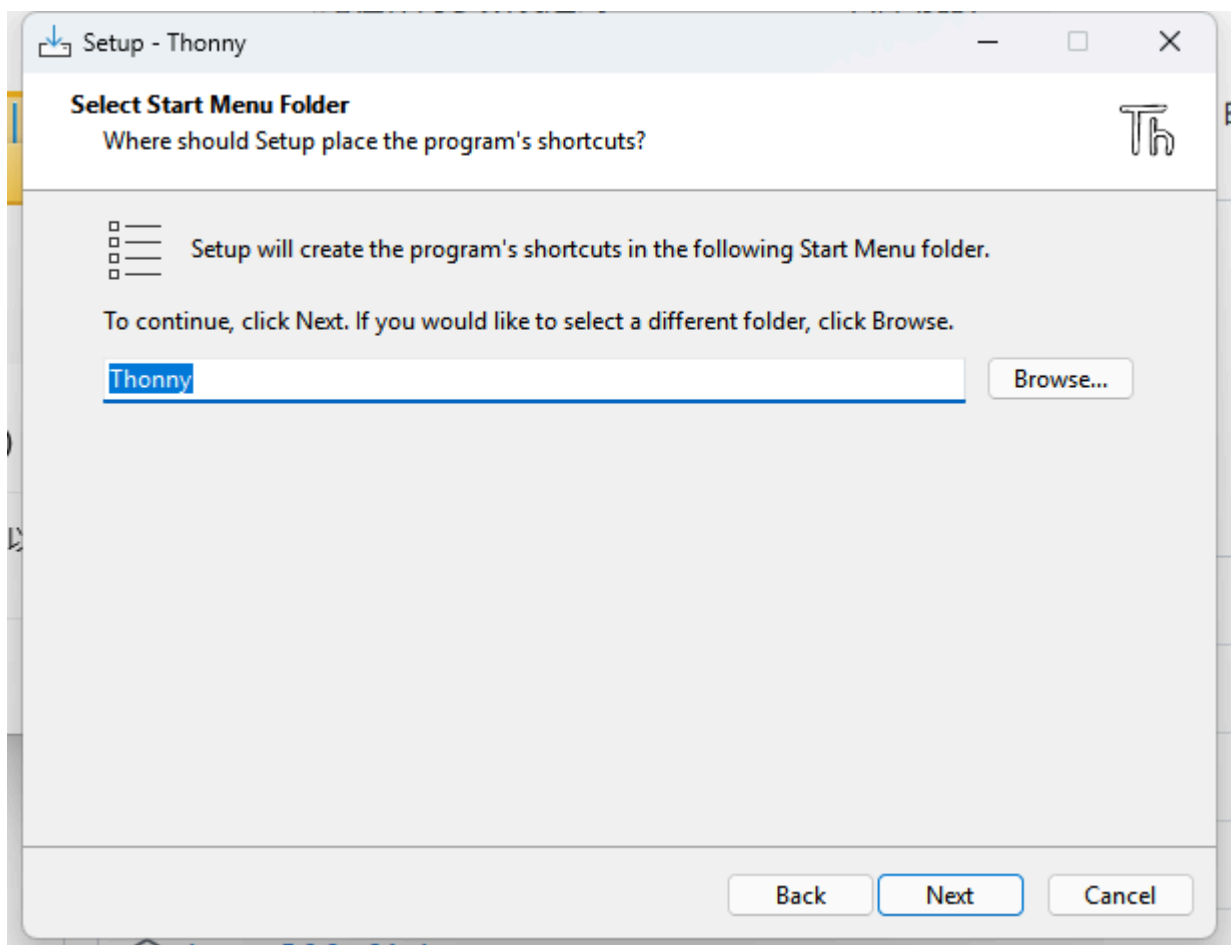
2. Click **Next**, read the license agreement, check **I accept the agreement**, click **Next**



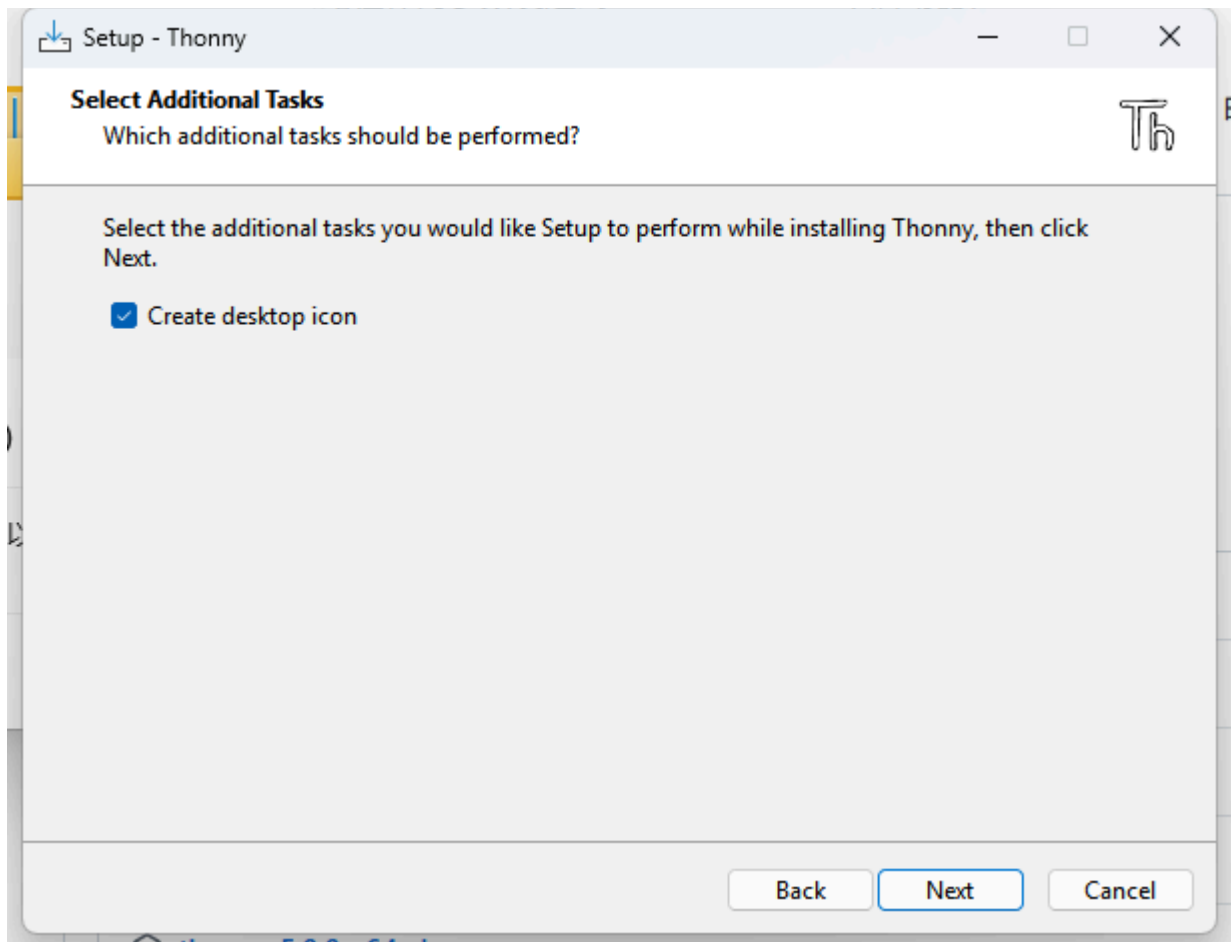
3. Select the installation path (keeping the default is recommended), click **Next**



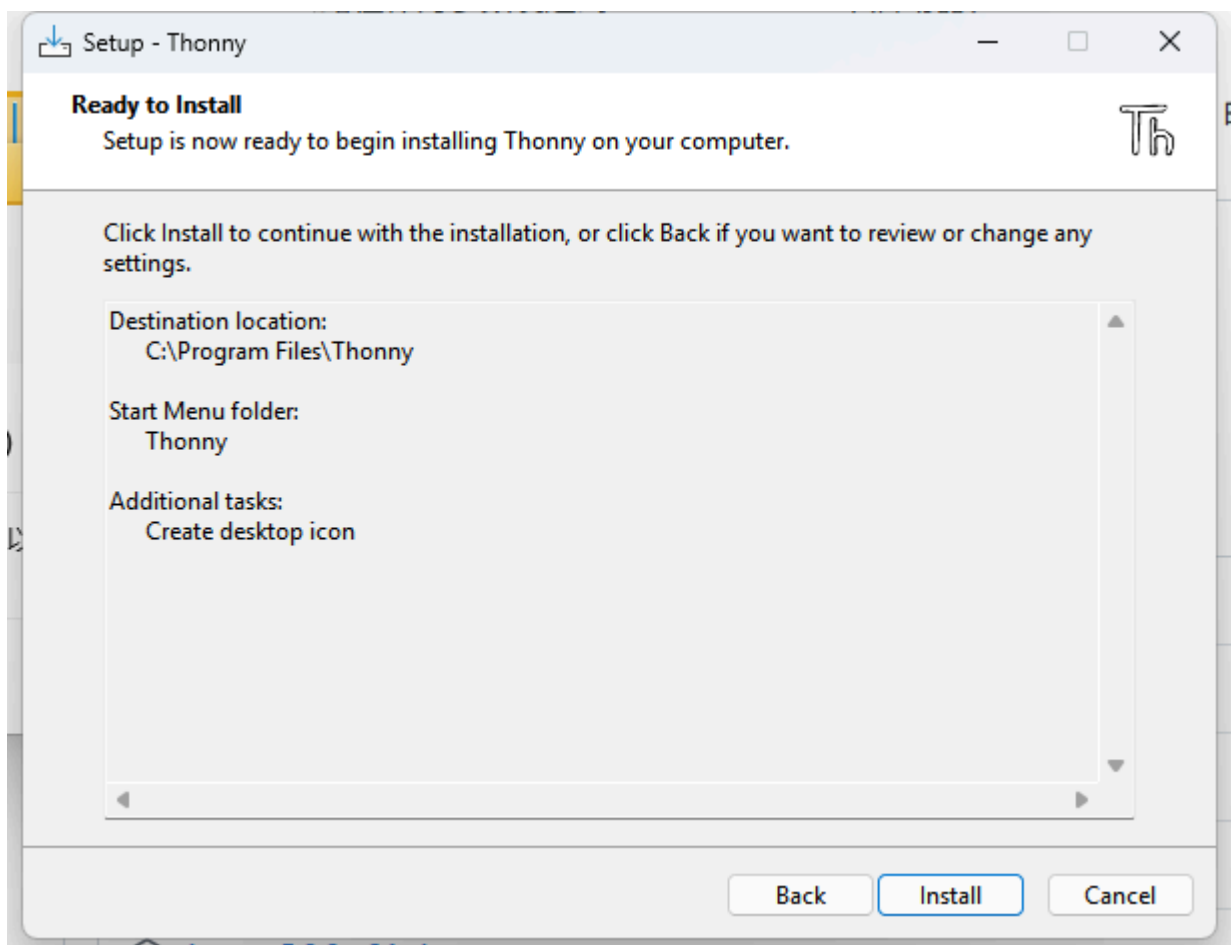
4. Select the Start Menu folder, click **Next**



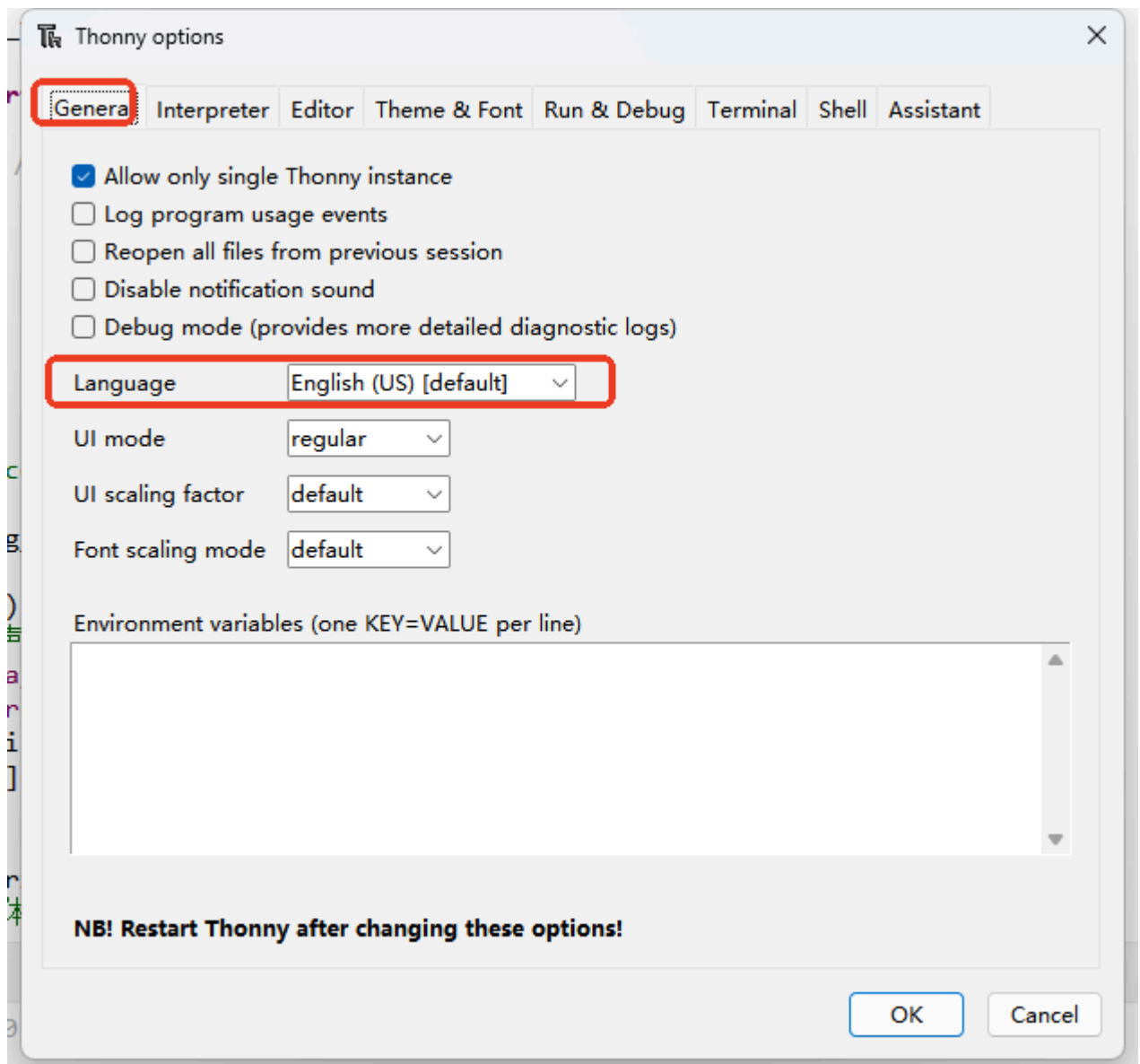
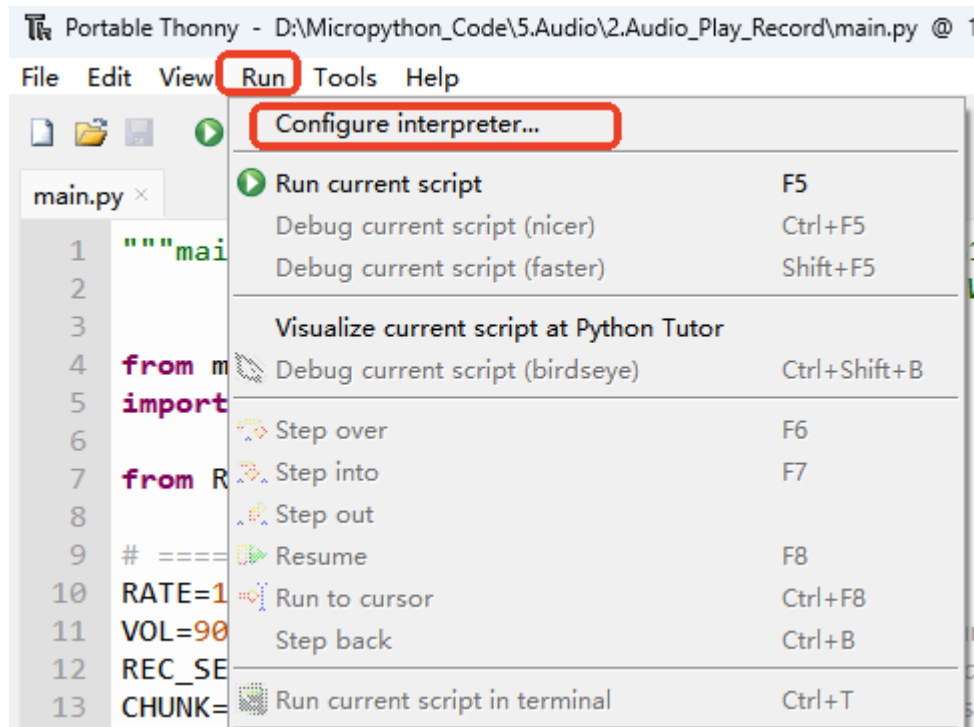
5. Check **Create desktop icon**, click **Next**



6. Click **Install**, wait for the installation to complete, click **Finish**



7. On first launch, you can choose the language as Simplified Chinese or keep English

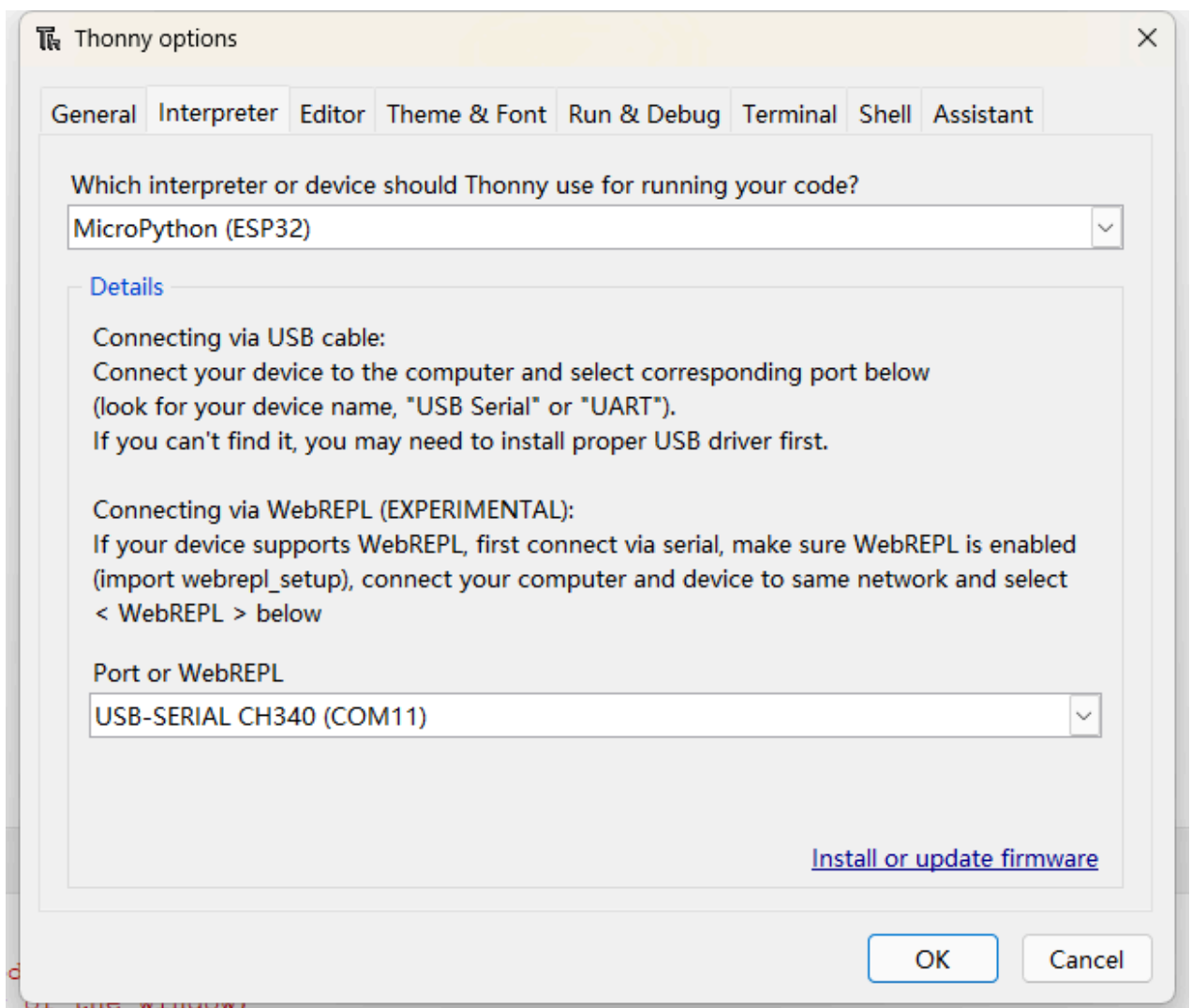


5.2 Connect the ESP32-S3

1. Connect the ESP32-S3 to the computer with a USB data cable
2. Open Thonny IDE and click the interpreter selection area in the bottom-right corner



3. In the pop-up menu, select **MicroPython (ESP32)**
4. In the **Port** dropdown below, select the COM port corresponding to the ESP32-S3



5. Click **OK** to confirm

Indicator of a successful connection: the **Shell** panel at the bottom shows the MicroPython REPL prompt:

```
MicroPython v1.28.0 on 2026-04-06; Generic ESP32S3 module with ESP32S3
Type "help()" for more information.
>>>
```

```
MicroPython v1.28.0 on 2026-04-06; Generic ESP32S3 module with ESP32S3
Type "help()" for more information.
>>>
```

If the Shell does not respond, press the **RST** key on the board to reset, or press **Ctrl + D** in the Shell panel for a soft reset.

6. Verification and the First Program

6.1 REPL Interaction Verification

Type Python statements directly after the `>>>` prompt and press Enter to execute:

```
>>> print("Hello ESP32-S3")
Hello ESP32-S3

>>> import machine
      machine.freq()
240000000

>>> import esp
      esp.flash_size()
16777216
```

```
>>> print("Hello ESP32-S3")
      Hello ESP32-S3

>>> import machine
      machine.freq()
240000000

>>> import esp
      esp.flash_size()
16777216

>>> |
```

- `print("Hello ESP32-S3")` — verifies basic Python execution
- `machine.freq()` — queries the CPU frequency (240MHz by default)
- `esp.flash_size()` — queries the Flash capacity (16MB = 16777216 bytes)

REPL (Read-Eval-Print Loop) is MicroPython's interactive console. Each line of code is interpreted and executed immediately, making it very suitable for quickly testing peripherals and debugging.

6.2 Run the First Script

1. Enter the following code in the Thonny editor area:

```
import machine
import time

led = machine.Pin(44, machine.Pin.OUT)

while True:
    led.value(1)
    time.sleep(0.5)
    led.value(0)
    time.sleep(0.5)
```

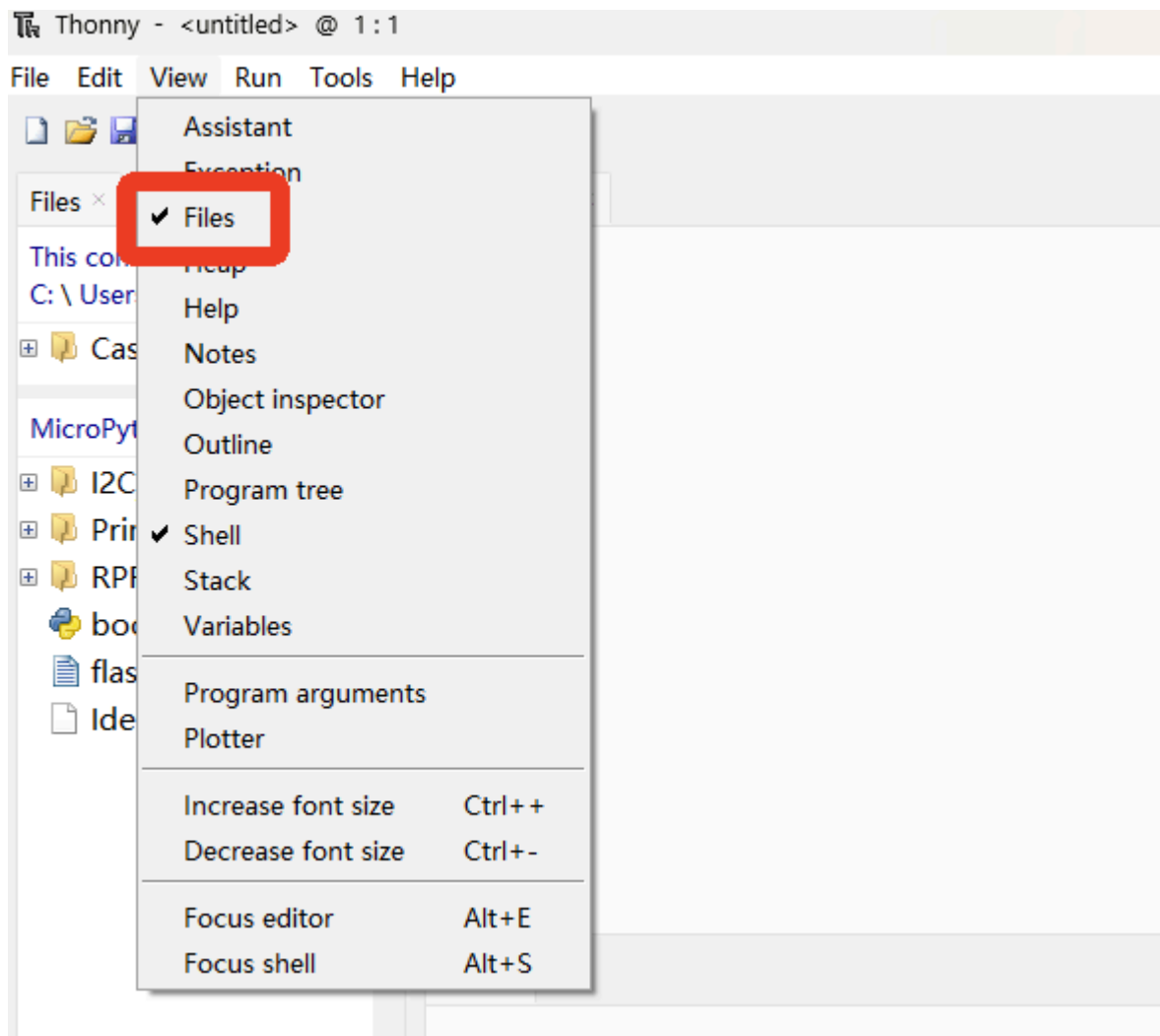
2. Click **Run** on the toolbar (the green  button) or press `F5`
3. A save dialog pops up; select **MicroPython device**, enter a file name (e.g., `blink.py`), click OK
4. The script is uploaded to the ESP32 and runs automatically; the on-board LED starts blinking at 0.5s intervals

Selecting **MicroPython device** saves the `.py` file to the ESP32-S3's Flash file system, which survives power loss. Selecting **This computer** only saves it locally.

6.3 File Management

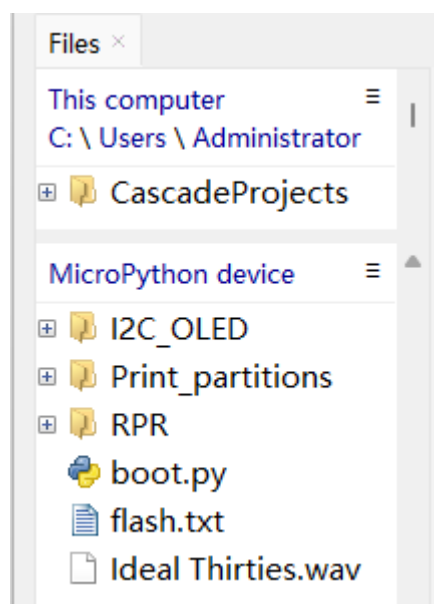
Thonny has a built-in file manager, so you can manage files on the ESP32 without extra tools:

1. Click **View** on the menu bar → check **Files**



2. The left panel automatically shows the file management area:

Panel	Path	Description
This computer	Local hard drive	Where local <code>.py</code> files and firmware files are stored
MicroPython device	<code>/flash</code>	ESP32-S3's internal file system (survives power loss)



3. Common operations:

- **Upload:** right-click a local file → **Upload to /** → copy to the ESP32
- **Download:** right-click a device file → **Download to /** → copy back to the computer
- **Delete:** right-click a device file → **Delete**

When the ESP32-S3 is powered on, it automatically executes `/flash/boot.py` (if present) and then `/flash/main.py` (if present). Naming your startup script `main.py` makes it run automatically on power-up.

7. Install Commonly Used Libraries

MicroPython uses the `mip` package manager to install third-party libraries; `pip` is not needed.

7.1 Online Installation via mip

In the Thonny Shell (REPL), run:

```
>>> import mip
>>> mip.install("umqtt.simple")    # MQTT client
>>> mip.install("urequests")      # HTTP client
```

Commonly used MicroPython libraries:

Library	Purpose	Later Experiment
<code>umqtt.simple</code>	MQTT client	MQTT communication experiment
<code>urequests</code>	HTTP client (requests-like)	HTTP client experiment
<code>ssd1306</code>	SSD1306 OLED driver (I2C/SPI)	I2C OLED display experiment
<code>neopixel</code>	WS2812 RGB LED driver	RGB colorful light experiment

`mip` requires the ESP32 to be connected to Wi-Fi. **Built-in firmware libraries** (`machine`, `time`, `network`, `socket`, etc.) need no installation; just `import` them directly.

7.2 Manually Upload .py Files

In an offline environment, you can download `.py` source files from [MicroPython Libs](#) and upload them to the device's `/flash/lib/` directory with the Thonny file manager:

1. Download the `.py` file of the target library to your computer
2. Create the `/flash/lib/` directory in the Thonny device panel (if it does not exist)
3. Right-click the `.py` file → **Upload to /flash/lib/**

`import` follows the standard Python path search rules: the device's `lib/` directory is in `sys.path`, so a direct `import` will find it.

8. Common Problems

Symptom	Cause	Solution
Flash fails with "Timed out waiting for packet header"	The chip did not enter download mode	Hold the BOOT key → press the EN key → release BOOT, retry flashing
Flash succeeds but REPL does not respond	Firmware does not match the chip	Make sure you downloaded an <code>ESP32_GENERIC_S3</code> series firmware, not <code>ESP32_GENERIC</code> (without S3)
<code>import machine</code> reports <code>no module named 'machine'</code>	Still using the local Python interpreter	Make sure the interpreter at the bottom-right of Thonny is set to MicroPython (ESP32)
Thonny cannot see the COM port	Missing driver, or the data cable does not support data transfer	Reinstall the CH340 driver; replace with a USB cable that supports data transfer
Garbled REPL output	Baud rate mismatch	Check in Thonny's interpreter options that the baud rate is 115200
Shell shows "busy" or freezes	The program is in an infinite loop occupying the REPL	Press Ctrl + C to interrupt the current program and return to the REPL
<code>mip.install</code> reports "OSError: -202"	Wi-Fi not connected	Connect to Wi-Fi in the REPL first, then run <code>mip</code>
Device files lost	File system accidentally formatted	If the file system is corrupted, run <code>import os; os.VfsLfs2.mkfs(bdev)</code> to reformat; files cannot be recovered
Out of memory (MemoryError)	The program uses too much memory	Use <code>gc.mem_free()</code> to check the remaining memory; for large arrays, store them in a file and read them on demand